



Cairo University
Faculty of Economics and Political Science
Socio-Computing Department
C201: Computer Applications in Social Sciences II

MODULE 2: PYTHON FUNDAMENTALS

LECTURE 2_4_PART2

PYTHON COLLECTION DATA TYPES:

TUPLES AND DICTIONARIES

DR. AMAL SAKR

OUTLINES

Python Fundamentals

Previous Lecture: *Python Collection Data Types - Lists*

This Lecture:

Python Collection Data Types- Tuples and Dictionaries

- Tuples: Definition and Creation, Access and Slice Tuples, Looping Over Tuples, and Tuple Methods
- Dictionaries: Definition and Creation, Access dictionary items, Looping Over Dictionary and Dictionary Methods

Tuples

- A tuple is virtually **identical to a list with one important difference – tuples are immutable.**
- *Immutability* means that a tuple cannot be changed once created. It is not possible to add, remove, or replace elements in a tuple.
- Tuples are constructed using parentheses **()** in place of the square brackets **[]** used to create lists.
- Tuples can contain any data type, and in any combination. So, a single tuple can contain strings, integers, floats, lists, dictionaries, and other tuples.
- *Accessing* and *slicing* tuple items are similar to the list data type.
- *Methods* related to tuples:
 - ✓ **tuple()**: to create an empty tuple
 - ✓ **count()**: to count the number of a specified item in a tuple
 - ✓ **index()**: to find the index of a specified item in a tuple
 - ✓ **+ operator**: to join two or more tuples and to create a new tuple

Access and Slice Tuples: *Example*

```
x = (0,10,20,30,40,50,60,70,80,90)
y = ('first', 'second', 'third')
z = (1871, 'Ahmed', 'Amer' , ['Math', 'Arabic'], (20, 30))
    #tuple of mixed types

type(z)          #Out: tuple

x = x + y
print(x , len(x))
#Out:(0, 10, 20, 30, 40, 50, 60,70, 80, 90, 'first', 'second', 'third'), 13

x[5:10]          #Out: (50, 60, 70, 80, 90)

x[0] = 7         #'tuple' object does not support item assignment

x = x[-10:];x
#Out: (30, 40, 50, 60, 70, 80, 90, 'first', 'second', 'third')

del x[11:14]      #'tuple' object does not support item deletion

y[-3:-1]         #Out: ('first', 'second')

y[-3:]           #Out: ('first', 'second', 'third')

x[: -3]          #Out: (30, 40, 50, 60, 70, 80, 90)

x= ([1,2],[3,4])

x[0][1] = 7; x    # Accepted operation Out: ([1, 7], [3, 4])
```

Looping Over a Tuple

The simplest and the most common way to iterate over a tuple is to use a **for loop**. Below is an example on how to iterate over a tuple using a for loop. It resembles iterating over both a list and a string.

```
print("Loop over a tuple")

numbers = (0, 1, 2, 3, 4, 5)

for number in numbers: print(number) #Out: 0 1 2 3 4 5

#----- Remember Looping over a List and a String -----

print("Loop over a list")

numbers = [0, 10, 20, 30, 40, 50]

for i in numbers: print(i) #Out: 0 10 20 30 40 50

print("Loop over a string")

language = "Python"

for letter in language: print(letter) #Out: P y t h o n
```

Looping Over a List of Tuples: *Example*

```
#Create a list of tuples, each name, age, and team position
team = [('Ahmed', 20, 'center'),
        ('Anas', 22, 'point guard'),
        ('Wael', 22, 'shooting guard'),
        ('Ameer', 21, 'power forward'),
        ('Ali', 19, 'small forward')]

#Print the name of the player, his position, and the team's average age
total=0

for name, age, position in team:
    print(name,position)
    total+=age

age_avg=total/len(team);print(age_avg)    #out: 20.8

print(team[1])          #out: ('Anas', 22, 'point guard')
print(team[1][2])       #out: 'point guard'
print(team[2:4])
#out: [('Wael', 22, 'shooting guard'), ('Ameer', 21, 'power forward')]
print(team[2:4][1])     #out: ('Ameer', 21, 'power forward')
```

Tuple Methods: *Example*

Python has two built-in methods that you can use on tuples:

- **Count()** which returns the number of times a specified value occurs in a tuple, and
- **Index()** which searches the tuple for a specified value and returns its position.

```
# Inflation rates (%) over 10 years
rates = (2.1, 1.8, 2.5, 3.0, 2.8, 1.9, 2.5, 3.2, 2.8, 2.5)

# Count occurrences of 2.5%
count_25 = rates.count(2.5)

# Find first year (index) with 3.0% inflation
first_30 = rates.index(3.0)

# Extract unique inflation rates
unique_rates = tuple(set(rates))

# Display results
print("Rates over 10 years:", rates)
print("2.5% occurred:", count_25, "times")
#Out: 2.5% occurred: 3 times

print("First 3.0% at year index:", first_30)
#Out: First 3.0% at year index: 3

print("Unique rates:", unique_rates)
#Out: Unique rates: (1.8, 2.1, 3.0, 2.5, 2.8, 1.9, 3.2)
```

Dictionary

A Python dictionary is a data structure that stores the value in **key: value** pairs. *Values* in a dictionary *can be of any data type* and *can be duplicated*, whereas *keys can't be repeated* and *must be immutable*.

To create a dictionary, we use curly brackets **{}** or the **dict()** built-in function. Dictionaries are commonly used to pass options into other functions.

```
# Example for creating a dictionary
```

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London" }
```

```
# printing the dictionary
```

```
print(country_capitals)
```

```
#Out: {'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}
```

Dictionaries are *changeable*, meaning that we can change, add or remove items after the dictionary has been created.

Dictionary

```
country_capitals = {  
    'Germany' : 'Berlin',  
    'Canada' : 'Ottawa',  
    'England' : 'London'  
}
```

Diagram illustrating the structure of a dictionary:

- element 1 points to the first pair: 'Germany' : 'Berlin'
- element 2 points to the second pair: 'Canada' : 'Ottawa'
- element 3 points to the third pair: 'England' : 'London'

Brackets below the dictionary indicate the components:

- key: points to the country names ('Germany', 'Canada', 'England')
- value: points to the capital cities ('Berlin', 'Ottawa', 'London')

- {'Germany', 'Canada', 'England'} are the *keys* (*index*) of this dictionary
- {'Berlin', 'Ottawa', 'London'} are the *values* of this dictionary
- {'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'} are the *items* (*elements*) of this dictionary

Creating a Dictionary

There are two main ways to create dictionaries in Python:

- Braces: {}
- The dict function: `dict()`

The `dict()` function uses a different syntax, where keys are entered as the function's keyword arguments and values are assigned with an equal's operator.

4 Different methods to create a dictionary

```
person_details_1 = dict(first_name="Rob",  
surname="Jone",gender="Male", favourite_food="Pizza")
```

```
personal_detail_2 = dict([("first name", "Rob"), ("surname",  
"Jone"), ("gender", "Male"), ("favourite_food", "Pizza")])
```

One list of tuples

```
person_details_3 = {"first_name": "Rob", "surname": "Jone",  
"gender": "Male", "favourite_food": "Pizza"}
```

```
personal_details_4 = {}      # or dict() create an empty dictionary
```

```
personal_details_4["first name"] = "Rob";
```

```
personal_details_4["surname"] = "Jone"
```

```
personal_details_4["gender"] = "Male";
```

```
personal_details_4["favourite_food"] = "Pizza"
```

Creating a Dictionary

Some important notes about keys and values:

- Dictionary keys: Can be of any immutable data type, such as strings, numbers, or tuples
- Dictionary values: Can be of any data type—mutable or immutable—including other dictionaries or objects
- Each key can only correspond to a single value; so, for example, this will throw an **error**:

```
invalid_dict = {'numbers': 1, 2, 3}
```

But if you enclose multiple values within another single data structure, you can create a valid dictionary.

For example:

```
valid_dict = {'numbers': [1, 2, 3]}
```

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries. Below are the most important ones

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary

Example

#Access and update dictionary items

```
pd= {} # or dict() create an empty dictionary
```

```
pd["first name"] = "Rob";
```

```
pd["surname"] = "Jone"
```

```
pd["gender"] = "Male";
```

```
pd["favourite_food"] = "Pizza"
```

pd.items()

```
#Out: dict_items([('first name', 'Rob'), ('surname',  
      'Jone'), ('gender', 'Male'), ('favourite_food', 'Pizza')])
```

pd.keys()

```
#Out: dict_keys(['first name', 'surname',  
      'gender', 'favourite_food'])
```

pd.values()

```
#Out: dict_values(['Rob', 'Jone', 'Male', 'Pizza'])
```

Example

#Access and update dictionary items

```
pd= {} # or dict() create an empty dictionary
```

```
pd["first name"] = "Rob";
```

```
pd["surname"] = "Jone"
```

```
pd["gender"] = "Male";
```

```
pd["favourite_food"] = "Pizza"
```

```
pd['first name']
```

```
#Access the value of a key Out: "Rob"
```

```
pd['hoppy']='tennis'
```

```
# Adding Items to a Dictionary
```

```
pd['first name']='James'
```

```
# update an item
```

```
print('hoppy' in pd)
```

```
# True.. Checking Keys in a Dictionary
```

```
pd.pop('favourite_food')
```

```
# removes an item
```

```
del pd['hoppy']
```

```
# removes an item
```

```
pd.popitem()
```

```
# removes the last item
```

```
print('gender' in pd) # False.. Check if a key exists in a dictionary
```

```
pd2 = pd.copy()
```

```
# Copy a Dictionary
```

```
pd.clear()
```

```
# clearing a Dictionary
```

```
del pd
```

```
# Deleting a Dictionary
```

Example

Write a Python program to create a dictionary that contains each character and its frequency in the following string **"Hello, I love feps"**.

The output should look like **{"H" : 1, "E" : 3,}**

```
txt = "Hello, I love feps"
txt_dic={}
L = set(txt.upper())

for i in L:
    txt_dic[i] = txt.count(i)

print(txt_dic)
out: {'O': 0, 'H': 1, 'E': 0, 'P': 0, 'F': 0, 'S': 0, ',': 1, ' ': 3, 'I': 1, 'L': 0, 'V': 0}
```

Access Complex Dictionary Values

Example

```
person = {'first_name': 'Mona',  
          'last_name': 'Saber',  
          'age': 250,  
          'country': 'Egypt',  
          'is_marred': True,  
          'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],  
          'address': {'street': 'kareem Banona', 'zipcode': '02210'}}
```

```
print("Loop over a dictionary"); i=0
```

```
for j in person: #by default person holds keys
```

```
    i+=1
```

```
    print(i,j) #access and print keys
```

```
person['skills']
```

```
    #out: ['JavaScript', 'React', 'Node', 'MongoDB', 'Python']
```

```
person['skills'][2] #out: 'Node'
```

```
person ['address']['street'] #out: 'Space street'
```

```
# --Output--  
1 first_name  
2 last_name  
3 age  
4 country  
5 is_marred  
6 skills  
7 address
```

Looping Over a Dictionary: *Example*

Write a Python program that analyzes marketing campaign performance and determines the best advertising platform based on Click-Through Rates (CTR).

```
# Function to find max and min low-performing ads platforms
def suggest_improvement(ctr_data, min_ctr=4.0):
    low_ctr = [p for p in ctr_data.keys() if ctr_data[p] < min_ctr]
    # or low_ctr = [p for p, c in ctr_data.items() if c < min_ctr]
    if low_ctr:
        print(f"\nPlatforms needing improvement CTR < {min_ctr}%:", low_ctr)
    else:
        print("\nAll platforms are performing well!")

# Ad campaign data (platform: CTR in %)
ad_ctr = {
    "Google Ads": 5.4,
    "Facebook Ads": 3.8,
    "Instagram Ads": 4.2,
    "LinkedIn Ads": 1.9 }
top= max(ad_ctr.values())
key = [k for k, v in ad_ctr.items() if v == top]
print(f"\nBest advertising platform: {key}")
suggest_improvement(ad_ctr)
suggest_improvement(ad_ctr,2)
```

```
# ----- Output -----
Best advertising platform: Google Ads
Platforms needing improvement (CTR < 4%):
['Facebook Ads', 'LinkedIn Ads']
Platforms needing improvement CTR < 2%:
['LinkedIn Ads']
```

Looping Over a List of Dictionaries: *Example*

```
#Analyzing a dataset of survey responses about their social media usage
def avg_hours(entries): # Function to get average hours spent
    return sum(e["hours"] for e in entries) / len(entries)

# Function to find heavy users (above threshold)
def heavy_users(entries, limit = 2):
    return [e["name"] for e in entries if e["hours"] > limit]
#-----
data = [
    {"name": "Alice", "age": 22, "platform": "Instagram", "hours": 3},
    {"name": "Bob", "age": 25, "platform": "Facebook", "hours": 1},
    {"name": "Charlie", "age": 19, "platform": "TikTok", "hours": 4},
    {"name": "David", "age": 30, "platform": "Twitter", "hours": 2} ]

for p in range(L): print (data[p])

avg = avg_hours(data)
print("\nAvg Hours:", round(avg, 2))

hu = heavy_users(data)
print("Heavy Users (greater than 2 hours):",hu)

hu = heavy_users(data, avg)
print(f"Heavy Users (greater than {avg} hours)", hu)
```

Summary

Feature	Tuple	List	Dictionary
Definition	Immutable ordered collection	Mutable ordered collection	Key-value pair collection
Example (Creation)	<code>gdp_growth = (2.5, 3.1, 2.8, 3.0)</code>	<code>countries = ['USA', 'UK', 'Germany', 'Canada']</code>	<code>population = {'USA': 331, 'UK': 68, 'Germany': 83}</code>
Mutability	✗ Immutable (cannot change values)	✓ Mutable (values can be changed)	✓ Mutable (values can be changed, keys are unique)
Indexing	✓ Supports indexing	✓ Supports indexing	✗ No positional indexing (access by key)
Duplicates	✓ Allows duplicates	✓ Allows duplicates	✗ Keys must be unique
Performance	✓ Faster (fixed size)	● Slower than tuple (dynamic)	● Slower (lookup optimization for key-value storage)
Use Case	When data should not change (e.g., fixed economic indicators)	When data needs modification (e.g., updating country lists)	When key-value mapping is needed (e.g., country-population data)
Common Methods	<code>count()</code> , <code>index()</code>	<code>append()</code> , <code>remove()</code> , <code>pop()</code> , <code>sort()</code> , <code>reverse()</code>	<code>keys()</code> , <code>values()</code> , <code>items()</code> , <code>get()</code> , <code>update()</code>

Sheet 4: Ex 4

1. Write a Python program to create a dictionary to store the below dataset

Product	Initial Price	new_price	initial_qty	new_qty
Laptop	1000	1500	50	20
Smartphone	800	1000	200	400
Headphones	100	80	500	700
Camera	1200	1100	30	35
Monitor	300	330	100	130

Then calculates the price elasticity of the demand using the below mathematical formula that :

$$Elasticity = \frac{\% \text{ Change in Quantity}}{\% \text{ Change in Price}} = \frac{(Q_2 - Q_1)/Q_1}{(P_2 - P_1)/P_1}$$

Determine if each product is elastic ($E > 1$), inelastic ($E < 1$), or unitary ($E = 1$)

Expected Output

Laptop: Elasticity = -1.2 → Inelastic

Smartphone: Elasticity = 4.0 → Elastic

Headphones: Elasticity = -2.0 → Inelastic

Camera: Elasticity = -2.0 → Inelastic

Monitor: Elasticity = 3.0 → Elastic

Answer

```
# Step 1: Create dictionary with new field names
```

```
products = {  
    "Laptop":      {"p0": 1000, "p1": 1500, "q0": 50,  "q1": 20},  
    "Smartphone": {"p0": 800,  "p1": 1000, "q0": 200, "q1": 400},  
    "Headphones": {"p0": 100,  "p1": 80,   "q0": 500, "q1": 700},  
    "Camera":      {"p0": 1200, "p1": 1100, "q0": 30,  "q1": 35},  
    "Monitor":     {"p0": 300,  "p1": 330,  "q0": 100, "q1": 130}  
}
```

Answer

Step 2: Calculate price elasticity for each product

```
print("Price Elasticity of Demand:\n")
```

```
for product, data in products.items():
```

```
    P0 = data["p0"];      P1 = data["p1"]
```

```
    Q0 = data["q0"];      Q1 = data["q1"]
```

```
# Percentage changes
```

```
price_change = (P1 - P0)/P0 ;   qty_change = (Q1 - Q0)/Q0
```

```
# Elasticity
```

```
    if price_change != 0:
```

```
        elasticity = round(qty_change / price_change,2)
```

```
    else:    elasticity = float('inf') #elasticity = infinity
```

```
# Classification
```

```
    if elasticity > 1:    category = "Elastic"
```

```
    elif elasticity < 1:    category = "Inelastic"
```

```
    else:    category = "Unitary"
```

```
# Output
```

```
    print(f"{product}: Elasticity = {elasticity} → {category}")
```